

Pemodelan Perangkat Lunak

Paramitha Nerisafitra, S.ST., M.Kom

Definisi

- Pemodelan (model) adalah representasi dari sistem yang disederhanakan.
- Macam-macam representasi sistem :

1. *Physical*

dengan membuat *scaled down version* dari sistem yang dipelajari (model pesawat, model kereta api)

2. *Pictorial*

representasi dengan gambar untuk menggambarkan kontur permukaan bumi seperti peta topografi dan bola dunia

Cont ...

3. *Verbal*

yaitu representasi suatu sistem ke dalam kalimat verbal yang menggambarkan ukuran, bentuk dan karakteristik

4. *Schematic*

yaitu representasi dalam bentuk skema figurative misalnya model rangkaian listrik, model Atom Bohr dan lain-lain

5. *Symbolic*

yaitu representasi ke dalam simbol-simbol matematik dimana variable hasil karakterisasi proses atau sistem ke dalam variable formulasi menggunakan simbol-simbol matematik

Proses pemodelan analisis

1. *Understandability*

yaitu sejauh mana proses secara eksplisit ditentukan dan bagaimana kemudahan definisi proses itu dimengerti.

2. *Visibility*

apakah aktivitas-aktivitas proses mencapai titik akhir dalam hasil yang jelas sehingga kemajuan dari proses Tersebut dapat terlihat nyata/jelas.

3. *Supportability*

yaitu sejauh mana aktivitas proses dapat didukung oleh CASE

4. *Acceptability*

apakah proses yang telah ditentukan oleh insinyur dapat diterima dan digunakan dan mampu bertanggung jawab selama pembuatan produk perangkat lunak

Proses pemodelan analisis

5. *Reliability*

apakah proses didesain sedikian rupa sehingga kesalahan proses dapat dihindari sebelum terjadi kesalahan pada produk.

6. *Robustness*

dapatkah proses terus berjalan walaupun terjadi masalah yang tak diduga.

7. *Maintainability*

dapatkah proses berkembang untuk mengikuti kebutuhan atau perbaikan.

8. *Rapidity*

bagaimana kecepatan proses pengiriman sistem dapat secara lengkap memenuhi spesifikasi.

Landskap pemodelan analisis



Analisis terstruktur



Analisis berorientasi objek

1. *Metode Booch*

Dikenal dengan nama *Metode Desain Object Oriented*.

Metode ini menjadikan proses analisis dan desain ke dalam empat tahapan yang iteratif (dapat berulang), yaitu :

1. identifikasi kelas-kelas dan objek-objek
 2. identifikasi semantik dan hubungan objek dan kelas tersebut
 3. perincian interface
 4. implementasi.
-

**Metodologi
berorientasi
objek**

2. Metode Rumbaugh (*Object Modelling Technique* - OMT)

Metode ini berdasarkan pada analisis terstruktur dan pemodelan *entity relationship*. Tahapan utama dalam metodologi ini adalah

1. Analisis
2. desain sistem dan desain objek
3. implementasi.

Keunggulan metode ini adalah dalam penotasian yang mendukung semua konsep *object oriented*.

Cont ...

Cont ...

3. Metode Jacobson (*Object Oriented Software Engineering* - OOSE)

Metode ini memberi penekanan lebih pada *use-case*.

OOSE memiliki tiga tahapan yaitu :

1. membuat model *requirement* dan analisis
2. desain dan implementasi
3. model pengujian (tes model).

Keunggulan metode ini adalah mudah untuk dipelajari karena memiliki notasi yang sederhana, mencakup seluruh tahapan dalam rekayasa *software*.

Cont ...

4. Metode Coad dan Yourdon

Metode ini didasarkan pada pemodelan *Object Oriented* dan *entity relationship*. Metode ini mempunyai perancangan yang berfokus pada empat komponen yaitu :

1. *Problem domain component*
2. *Human interaction component*
3. *Data management component*
4. *Task management component*

Cont ...

5. Metode Wirfs-Brock

- ❑ *Responsibility Driven Design/ Class Responsibility Collaboration (RDD/CFC)*
- ❑ Metode ini diarahkan pada desain, tetapi sangat berguna untuk memunculkan ide dalam tahap analisis.
- ❑ Keunggulannya adalah mudah digunakan, metode ini juga mengidentifikasikan hirarki kelas dan subsistem-subsistem.

Cont ...

6. Metode Shlair - *Mellor Object Oriented Analysis/Design (OOA/D)*

- ❑ Metode yang menggunakan teknik pemodelan informasi tradisional yang menjelaskan entitas dalam sistem.
- ❑ Metode ini menghasilkan tiga jenis model yaitu
 - *information model*
 - *State model*
 - *process model.*
- ❑ Keunggulan metode ini adalah dalam memandang masalah dari sudut pandang yang berbeda, mudah dibuat (dikonversi) dari metode struktural.

Unified Modeling Language (UML)

- *Unified Modeling Language* (UML) adalah notasi yang lengkap untuk membuat visualisasi model suatu sistem
- Tujuan *Unified Modeling Language* (UML) diantaranya adalah :
 1. Memberikan model yang siap pakai, bahasa pemodelan visual yang ekspresif untuk mengembangkan sistem dan yang dapat saling menukar model dengan mudah dan dimengerti secara umum.
 2. Memberikan bahasa pemodelan yang bebas dari berbagai bahasa pemrograman dan proses rekayasa.
 3. Menyatukan praktek – praktek terbaik yang terdapat dalam pemodelan.

1. Use case diagram

- *Use Case diagram* merupakan suatu diagram yang menggambarkan fungsionalitas yang diharapkan dari sebuah sistem
- Sebuah *use case* dapat memrepresentasikan interaksi antara aktor dengan sistem
- *Use case* bekerja dengan cara mendeskripsikan tipe interaksi antara user sebuah system dengan systemnya sendiri melalui sebuah cerita bagaimana sebuah system dipakai

6 bagian dari *use case*

1. *System*

- Menyatakan batasan sistem dalam relasi dengan actor-actor yang menggunakannya (di luar sistem) dan fitur-fitur yang harus disediakan (dalam sistem).
- Digambarkan dengan segi empat yang membatasi semua use case dalam sistem terhadap pihak mana sistem akan berinteraksi.
- Sistem disertai label yang menyebutkan nama dari sistem, tapi umumnya tidak digambarkan karena tidak terlalu memberi arti tambahan pada diagram.

6 bagian dari *use case*

2. *Actor*

- Aktor adalah segala hal diluar sistem yang akan menggunakan sistem tersebut untuk melakukan sesuatu.
- Bisa merupakan manusia, sistem, atau device yang memiliki peranan dalam keberhasilan operasi dari sistem.
- Cara mudah untuk menemukan aktor adalah dengan bertanya hal-hal berikut :
 1. SIAPA yang akan menggunakan sistem?
 2. APAKAH sistem tersebut akan memberikan NILAI bagi aktor?

6 bagian dari *use case*

3. *Use Case*

- Mengidentifikasi fitur kunci dari sistem. Tanpa fitur ini, sistem tidak akan memenuhi permintaan user/actor.
- Setiap use case mengekspresikan goal dari sistem yang harus dicapai. Diberi nama sesuai dengan goal-nya dan digambarkan dengan elips dengan nama di dalamnya.
- Fokus tetap pada goal bukan bagaimana mengimplementasikannya walaupun use case berimplikasi pada prosesnya nanti. Setiap use case biasanya memiliki trigger/pemicu yang menyebabkan use case memulai (misalnya, Pasien mendaftar dan membuat janji baru atau meminta untuk membatalkan atau mengubah janji yang sudah ada)
- Ada 2 trigger pertama trigger eksternal, seperti pelanggan memesan atau alarm kebakaran berbunyi, kedua trigger temporal, seperti tanggal pengembalian buku terlewat di perpustakaan atau keterlambatan bayar sewa.

6 bagian dari *use case*

4. *Association*

- Mengidentifikasi interaksi antara setiap actor tertentu dengan setiap use case tertentu.
- Digambarkan sebagai garis antara actor terhadap use case yang bersangkutan.
- Asosiasi bisa berarah (garis dengan anak panah) jika komunikasi satu arah, namun umumnya terjadi kedua arah (tanpa anak panah) karena selalu diperlukan demikian.

6 bagian dari *use case*

5. *Dependency*

Dependensi << include >>

1. Mengidentifikasi hubungan antar dua use case di mana yang satu memanggil yang lain.
2. Jika pada beberapa use case terdapat bagian yang memiliki aktivitas yang sama maka bagian aktivitas tersebut biasanya dijadikan use case tersendiri dengan relasi dependensi setiap use case semula ke use case yang baru ini sehingga memudahkan pemeliharaan.
3. Digambarkan dengan garis putus-putus bermata panah dengan notasi <<include>> pada garis.
4. Arah mata panah sesuai dengan arah pemanggilan.

6 bagian dari *use case*

5. *Dependency*

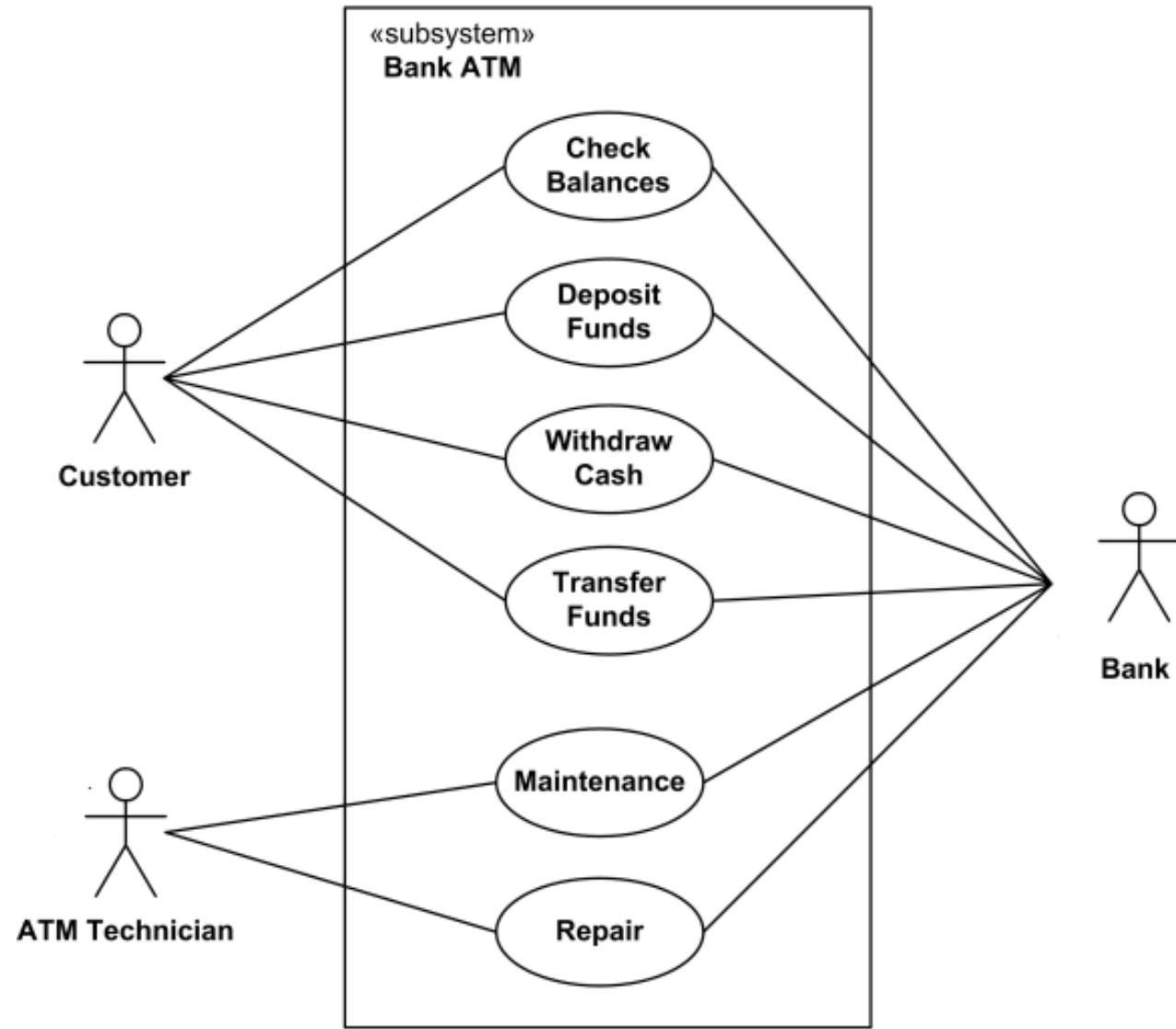
Dependensi << extend >>

1. Jika pemanggilan memerlukan adanya kondisi tertentu maka berlaku dependensi <<extend>>.
2. Note: konsep “extend” ini berbeda dengan “extend” dalam Java!
3. Digambarkan serupa dengan dependensi <<include>> kecuali arah panah berlawanan

6 bagian dari *use case*

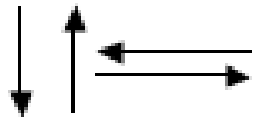
6. *Generalization*

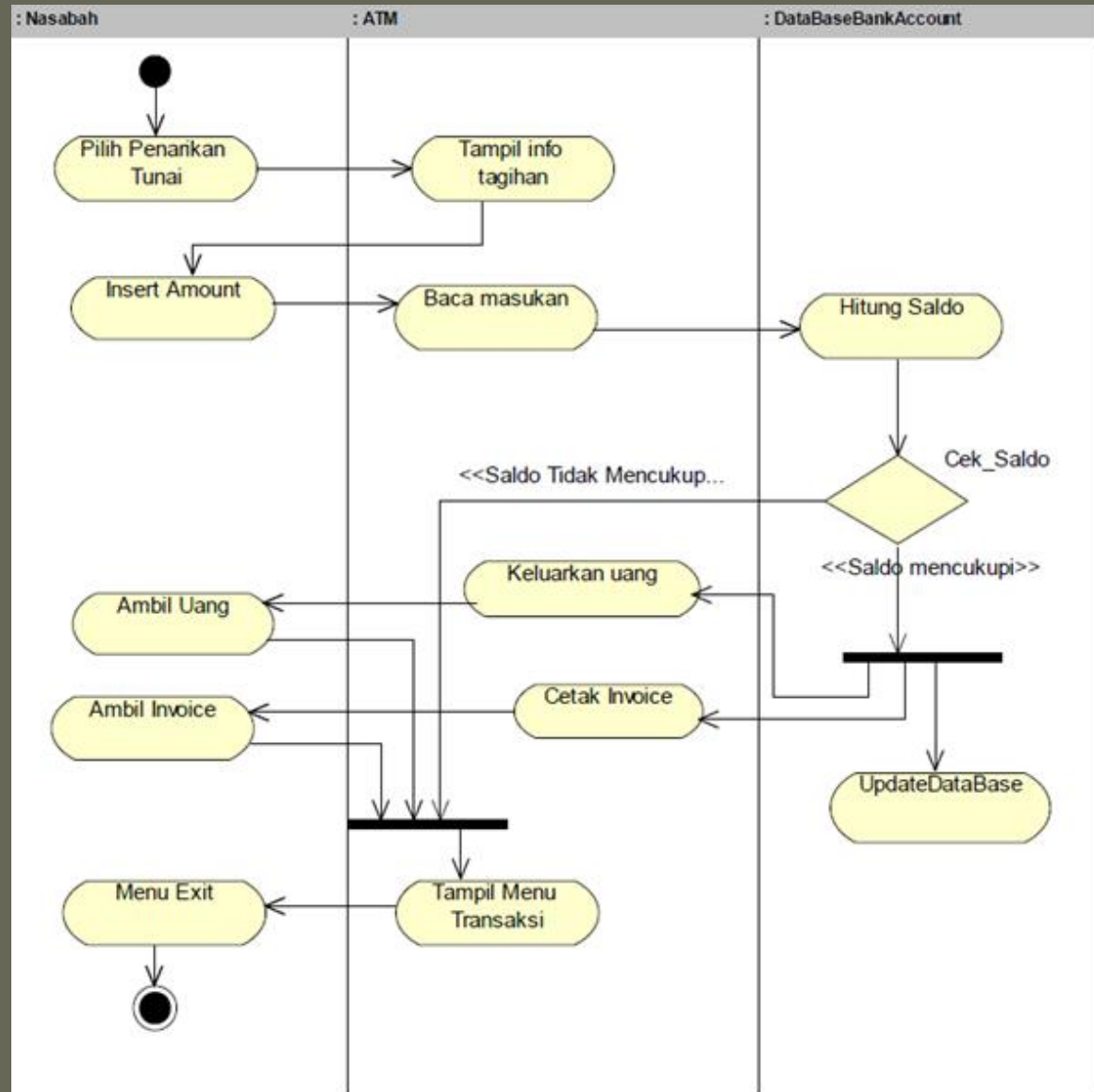
Mendefinisikan relasi antara dua actor atau dua use case yang mana salah satunya meng-inherit dan menambahkan atau override sifat dari yang lainnya. Penggambaran menggunakan garis bermata panah kosong dari yang meng-inherit mengarah ke yang di-inherit



2. Activity Diagram

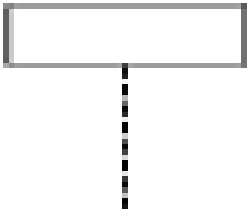
- ❑ Activity diagram menggambarkan berbagai alur aktivitas dalam sistem yang sedang dirancang, bagaimana masing-masing alur berawal, decision yang mungkin terjadi, dan bagaimana mereka berakhir.
- ❑ Activity diagram juga dapat menggambarkan proses paralel yang mungkin terjadi pada beberapa eksekusi.
- ❑ Activity diagram merupakan state diagram khusus, di mana sebagian besar state adalah action dan sebagian besar transisi di-trigger oleh selesainya state sebelumnya (internal processing). Oleh karena itu activity diagram tidak menggambarkan behaviour internal sebuah sistem (dan interaksi antar subsistem) secara eksak, tetapi lebih menggambarkan proses-proses dan jalur-jalur aktivitas dari level atas secara umum.

NO	GAMBAR	NAMA	KETERANGAN
1		<i>Activity</i>	Memperlihatkan bagaimana masing-masing kelas antarmuka saling berinteraksi satu sama lain
2		<i>Action</i>	State dari sistem yang mencerminkan eksekusi dari suatu aksi
3		<i>Initial Node</i>	Bagaimana objek dibentuk atau diawali.
4		<i>Activity Final Node</i>	Bagaimana objek dibentuk dan diakhiri
5		<i>Decision</i>	Digunakan untuk menggambarkan suatu keputusan / tindakan yang harus diambil pada kondisi tertentu
6		<i>Line Connector</i>	Digunakan untuk menghubungkan satu simbol dengan simbol lainnya



3. Sequence Diagram

- ❑ Sequence diagram menggambarkan interaksi antar objek di dalam dan di sekitar sistem (termasuk pengguna, display, dan sebagainya) berupa message yang digambarkan terhadap waktu.
- ❑ Sequence diagram terdiri atas dimensi vertikal (waktu) dan dimensi horizontal (objek-objek yang terkait).
- ❑ Sequence diagram dapat digunakan untuk menggambarkan skenario atau rangkaian langkah-langkah yang dilakukan sebagai respons dari sebuah event untuk menghasilkan output tertentu.

NO	GAMBAR	NAMA	KETERANGAN
1		<i>LifeLine</i>	Objek <i>entity</i> , antarmuka yang saling berinteraksi.
2		<i>Message</i>	Spesifikasi dari komunikasi antar objek yang memuat informasi-informasi tentang aktifitas yang terjadi
3		<i>Message</i>	Spesifikasi dari komunikasi antar objek yang memuat informasi-informasi tentang aktifitas yang terjadi

:Operator

:ATM

:CashDispenser

:Jaringanke Bank

